



G L O B A L R A I N

Practices for Secure Software Report

Table of Contents

TABLE OF CONTENTS	2
ALGORITHM CIPHER	4
CERTIFICATE GENERATION.....	5
DEPLOY CIPHER	5
SECURE COMMUNICATIONS	6
SECONDARY TESTING	7
FUNCTIONAL TESTING	9
SUMMARY	10
INDUSTRY STANDARD BEST PRACTICES	11
REFERENCES	12
APPENDIX A: REFACTORED CODE FILES.....	13

Document Revision History

Version	Date	Author	Comments
1.0	08/16/2026	Erica Kinch	Final

Client



Developer

Erica Kinch

Algorithm Cipher

Artemis Financial needs both encryption and hashing because these security measures serve different purposes. I recommend AES-256-GCM as the encryption cipher for protecting sensitive financial information. AES is a symmetric block cipher, meaning the same secret key is used to encrypt and decrypt data. It processes information in 128-bit blocks and supports keys of 128, 192, or 256 bits. The 256-bit key provides a large key space, while Galois/Counter Mode provides confidentiality and authentication. This allows the system to encrypt information and detect whether the encrypted content has been altered (National Institute of Standards and Technology [NIST], 2001; Dworkin, 2007).

AES-256-GCM requires a unique initialization vector, sometimes called a nonce, for each encryption operation performed with the same key. The initialization vector does not need to remain secret, but it should not be reused with the same key. A secure random-number generator should be used to create these values. Reusing a GCM initialization vector can weaken the authentication protections and expose encrypted information. The AES key must also be stored separately from the encrypted data and protected from unauthorized access.

Symmetric algorithms such as AES are efficient for encrypting large amounts of application data. Asymmetric algorithms use a mathematically related public and private key pair. In this project, RSA was used to generate the key pair for the server certificate. The public key can be shared, while the private key must remain protected. During TLS communication, asymmetric cryptography helps establish server identity and secure key exchange, while symmetric encryption protects the transmitted session data.

For the required checksum, I selected the SHA-256 cryptographic hash algorithm. SHA-256 is not an encryption cipher because it does not use a key and its output cannot be decrypted. It processes data of any length and produces a fixed 256-bit message digest, normally displayed as 64 hexadecimal characters. The same input produces the same digest, but modifying the input changes the result. This makes SHA-256 suitable for detecting unexpected changes to Artemis Financial's data (NIST, 2015).

Encryption standards have changed as computing power and cryptanalysis have advanced. DES is no longer appropriate because its 56-bit key is vulnerable to brute-force attacks. Triple DES extended the useful life of DES but is slower and has also been retired from many modern uses. MD5 and SHA-1 are no longer appropriate for collision-resistant security applications. AES remains a current standard for symmetric encryption, while SHA-256 remains an accepted member of the SHA-2 family for integrity verification. Java supports SHA-256 through the MessageDigest API using the standard name "SHA-256" (Oracle, 2017).

Certificate Generation

I generated a self-signed X.509 certificate with Java's keytool utility. The certificate uses a 2,048-bit RSA public and private key pair and is stored in a PKCS12 keystore named keystore.p12. I assigned the certificate the alias sslserver and used localhost as its common name. I also included subject alternative names for localhost and 127.0.0.1 so the certificate would identify the local addresses used to test the application.

The keytool -genkeypair command creates a public and private key pair and places the public key in a self-signed X.509 certificate when a separate signer is not provided (Oracle, n.d.). The private key remains protected inside the keystore. I exported the public certificate as sslserver.cer and verified its owner, issuer, validity period, fingerprint, signature algorithm, public-key algorithm, and subject alternative names.

A self-signed certificate is appropriate for demonstrating TLS communication in a local development environment. Because a trusted certificate authority did not issue it, the browser displays a certificate warning. A production system should use a certificate issued by a trusted certificate authority so clients can verify the server's identity.

```
● erica@Ericas-MacBook-Air ssl-server_student % keytool -printcert -file src/main/resources/sslserver.cer

Owner: CN=localhost, OU=CS 305, O=SNHU, L=Horseheads, ST=NY, C=US
Issuer: CN=localhost, OU=CS 305, O=SNHU, L=Horseheads, ST=NY, C=US
Serial number: 8cde0c6bd1bd4c2e
Valid from: Tue Aug 11 19:40:49 EDT 2026 until: Fri Aug 08 19:40:49 EDT 2036
Certificate fingerprints:
    SHA1: D0:55:BF:9D:1B:37:31:B5:DA:01:55:D5:24:75:CC:94:E1:67:18:8C
    SHA256: 81:E8:43:0E:BD:50:F9:28:27:67:CC:48:1A:A3:D2:2F:10:6E:ED:40:59:9F:F1:D1:D1:19:7E:33:B3
:41:0E:CF
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.17 Criticality=false
SubjectAlternativeName [
  DNSName: localhost
  IPAddress: 127.0.0.1
]

#2: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
  KeyIdentifier [
    0000: 9D 1A 37 06 DA EC CC 0B 22 E4 C2 07 20 2A 50 19 ..7.....".... *P.
    0010: 0F CF 78 36 ..x6
  ]
]

○ erica@Ericas-MacBook-Air ssl-server_student %
```

Figure 1: Self-signed X.509 certificate generated for the SSL server.

Deploy Cipher

I deployed the SHA-256 hashing algorithm in the `SslServerApplication` class by creating a `/hash` endpoint. When the endpoint receives a request, the `getHash()` method assigns the static string “Erica - Artemis Financial Secure Data 2026” to the data variable. The method then creates a `MessageDigest` instance by calling `MessageDigest.getInstance("SHA-256")`.

The application converts the static string into bytes using `StandardCharsets.UTF_8`. Specifying UTF-8 prevents the checksum from depending on the computer’s default character encoding. The `digest()` method processes the encoded data and returns the SHA-256 digest as a byte array. A loop converts each byte into hexadecimal form and adds a leading zero when necessary to preserve the correct two-character representation of each byte.

The application returns the original string and its 64-character SHA-256 checksum. The generated checksum was `32cd22ac9ee9f135bc77328bfe505a6324b381fb6a918300cc6beb25894aa7df`. Repeating the operation with the same data produces the same value. A change to the original string would produce a different digest, providing a method for detecting whether the information was modified.

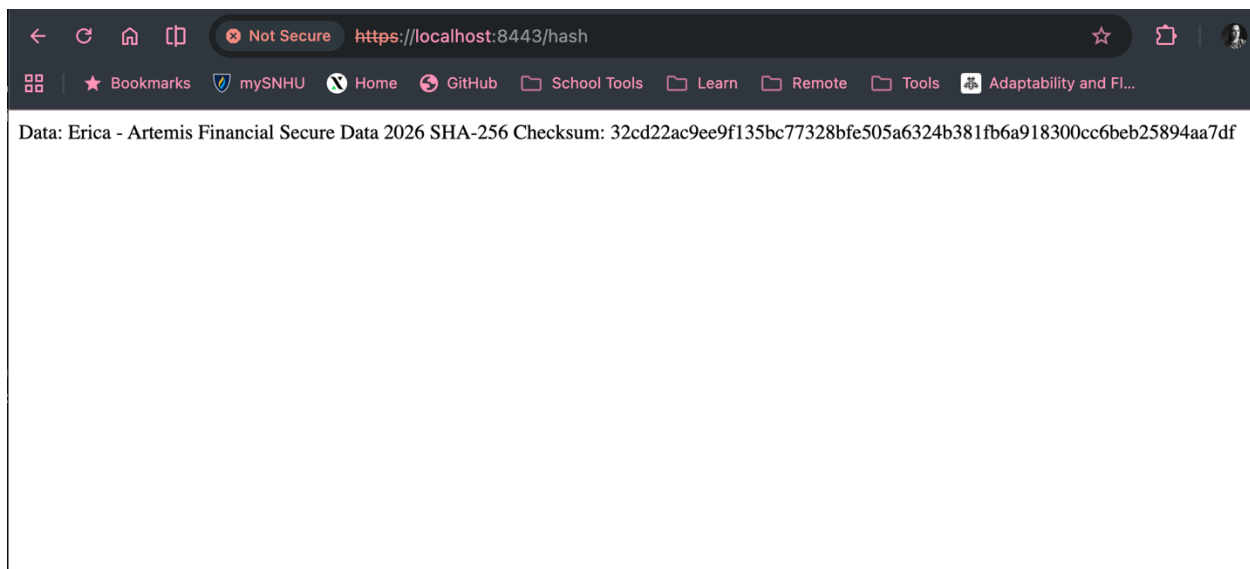


Figure 2: Static application data and the SHA-256 checksum returned by the `/hash` endpoint.

Secure Communications

I configured the application to use HTTPS instead of HTTP so information transmitted between the server and client would be encrypted. In `application.properties`, I changed the server port to 8443, enabled SSL, identified `sslserver` as the certificate alias, and configured Spring Boot to load the PKCS12 keystore from the application’s classpath. The keystore contains the private key and self-signed certificate needed to establish the TLS connection.

I verified the connection by opening `https://localhost:8443/hash` in the browser. The HTTPS scheme and port 8443 confirm that the endpoint was delivered through TLS instead of unencrypted HTTP. The browser displayed “Not Secure” because the certificate was self-signed and was not issued by a certificate authority trusted by the browser. This warning is expected during local testing and does not mean that HTTPS was disabled.

The TLS configuration adds confidentiality for data in transit, while the SHA-256 checksum provides an integrity check for the returned data. For production use, Artemis Financial should obtain a certificate from a trusted certificate authority and redirect all HTTP traffic to HTTPS.

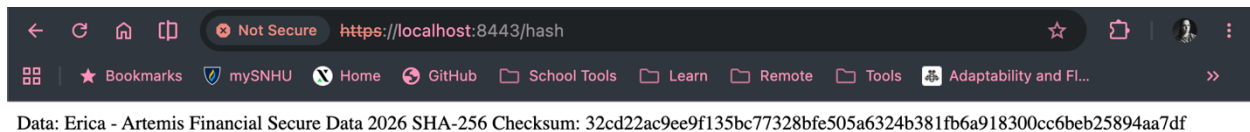


Figure 3: The application’s /hash endpoint accessed through HTTPS on port 8443.

Secondary Testing

I completed secondary static testing with OWASP Dependency-Check version 12.1.0. Dependency-Check is a software composition analysis tool that examines third-party components and associates identified packages with published vulnerability records (OWASP Foundation, 2026). The initial scan examined 49 dependencies and reported 15 vulnerable dependencies with 242 findings. Many of the findings came from the outdated Spring Boot 2.2.4 dependency tree.

I reviewed the report and updated the project’s dependencies. I upgraded Spring Boot to version 4.1.0, changed the project to Java 17, removed the unused Spring Data REST dependency, and updated the managed Tomcat and Log4j versions. I rebuilt the application and reran Dependency-Check after making these changes.

The updated scan initially contained one finding involving CVE-2026-66299 and Tomcat 11.0.24. The reported condition concerned the optional Tomcat WebSocket chat example. That example application was not included or deployed in this Spring Boot project, so the affected functionality was not present. I documented this finding with a narrowly scoped suppression that applies only to CVE-2026-66299 and the identified Tomcat components.

The final report scanned 39 dependencies and showed zero vulnerable dependencies, zero active vulnerabilities, and one documented suppression. Figure 6 also shows that the refactored application



DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) |
 [Suppressing false positives](#) |
 [Getting Help: github issues](#)

[Sponsor](#)

Project: ssl-server

com.snhu:ssl-server:0.0.1-SNAPSHOT

Scan Information ([show all](#)):

- *dependency-check version:* 12.1.0
- *Report Generated On:* Tue, 11 Aug 2026 20:08:52 -0400
- *Dependencies Scanned:* 49 (31 unique)
- *Vulnerable Dependencies:* 15
- *Vulnerabilities Found:* 242
- *Vulnerabilities Suppressed:* 0
- ...

Summary

Display: [Showing Vulnerable Dependencies \(click to show all\)](#)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
hibernate-validator-6.0.18.Final.jar	cpe:2.3.a.hibernate.hibernate-validator:6.0.18:*:*:*:*:* cpe:2.3.a.redhat.hibernate_validator:6.0.18:*:*:*:*:* cpe:2.3.a.validator.validator:6.0.18:*:*:*:*:*	pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final	MEDIUM	4	Highest	32
jackson-databind-2.10.2.jar	cpe:2.3.a.fasterxml.jackson-core:2.10.2:*:*:*:*:* cpe:2.3.a.fasterxml.jackson-databind:2.10.2:*:*:*:*: cpe:2.3.a.fasterxml.jackson-modules-java8:2.10.2:*:*:*:*:	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.10.2	HIGH	11	Highest	39
json-path-2.4.0.jar	cpe:2.3.a.json-path.jayway_jsonpath:2.4.0:*:*:*:*:	pkg:maven/com.jayway.jsonpath/json-path@2.4.0	MEDIUM	1	Highest	33
json-smart-2.3.jar	cpe:2.3.a.json-smart_project.json-smart:2.3:*:*:*:*: cpe:2.3.a.json-smart_project.json-smart-v2:2.3:*:*:*:*:	pkg:maven/net.minidev/json-smart@2.3	HIGH	2	Highest	45
log4j-api-2.12.1.jar	cpe:2.3.a.apache.log4j:2.12.1:*:*:*:*:	pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1	MEDIUM	5	Highest	42
logback-core-1.2.3.jar	cpe:2.3.a.qos.logback:1.2.3:*:*:*:*:	pkg:maven/ch.qos.logback/logback-core@1.2.3	HIGH	2	Highest	31
snakeyaml-1.25.jar	cpe:2.3.a.snakeyaml_project.snakeyaml:1.25:*:*:*:*:	pkg:maven/org.yaml/snakeyaml@1.25	CRITICAL	8	Highest	44
spring-boot-2.2.4.RELEASE.jar	cpe:2.3.a.vmware.spring_boot:2.2.4:release:*:*:*:	pkg:maven/org.springframework.boot/spring-boot@2.2.4.RELEASE	CRITICAL	8	Highest	39
spring-boot-starter-web-2.2.4.RELEASE.jar	cpe:2.3.a.vmware.spring_boot:2.2.4:release:*:*:*: cpe:2.3.a.web_project.web:2.2.4:release:*:*:*:	pkg:maven/org.springframework.boot/spring-boot-starter-web@2.2.4.RELEASE	CRITICAL	8	Highest	35
spring-core-5.2.3.RELEASE.jar	cpe:2.3.a.pivotal_software.spring_framework:5.2.3:release:*:*:*: cpe:2.3.a.springsource.spring_framework:5.2.3:release:*:*:*: cpe:2.3.a.vmware.spring_framework:5.2.3:release:*:*:*:	pkg:maven/org.springframework/spring-core@5.2.3.RELEASE	CRITICAL*	18	Highest	36
spring-hateoas-1.0.3.RELEASE.jar	cpe:2.3.a.vmware.spring_hateoas:1.0.3:release:*:*:*:	pkg:maven/org.springframework.hateoas/spring-hateoas@1.0.3.RELEASE	MEDIUM	1	Highest	43
spring-web-5.2.3.RELEASE.jar	cpe:2.3.a.pivotal_software.spring_framework:5.2.3:release:*:*:*: cpe:2.3.a.springsource.spring_framework:5.2.3:release:*:*:*: cpe:2.3.a.vmware.spring_framework:5.2.3:release:*:*:*: cpe:2.3.a.web_project.web:5.2.3:release:*:*:*:	pkg:maven/org.springframework/spring-web@5.2.3.RELEASE	CRITICAL*	19	Highest	34
spring-webmvc-5.2.3.RELEASE.jar	cpe:2.3.a.pivotal_software.spring_framework:5.2.3:release:*:*:*: cpe:2.3.a.springsource.spring_framework:5.2.3:release:*:*:*: cpe:2.3.a.vmware.spring_framework:5.2.3:release:*:*:*: cpe:2.3.a.web_project.web:5.2.3:release:*:*:*:	pkg:maven/org.springframework/spring-webmvc@5.2.3.RELEASE	CRITICAL*	18	Highest	36
tomcat-embed-core-9.0.30.jar	cpe:2.3.a.apache.tomcat:9.0.30:*:*:*:*: cpe:2.3.a.apache_tomcat.apache_tomcat:9.0.30:*:*:*:	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@9.0.30	CRITICAL*	68	Highest	30
tomcat-embed-websocket-9.0.30.jar	cpe:2.3.a.apache.tomcat:9.0.30:*:*:*:*: cpe:2.3.a.apache_tomcat.apache_tomcat:9.0.30:*:*:*:	pkg:maven/org.apache.tomcat.embed/tomcat-embed-websocket@9.0.30	CRITICAL*	69	Highest	30

* indicates the dependency has a known exploited vulnerability

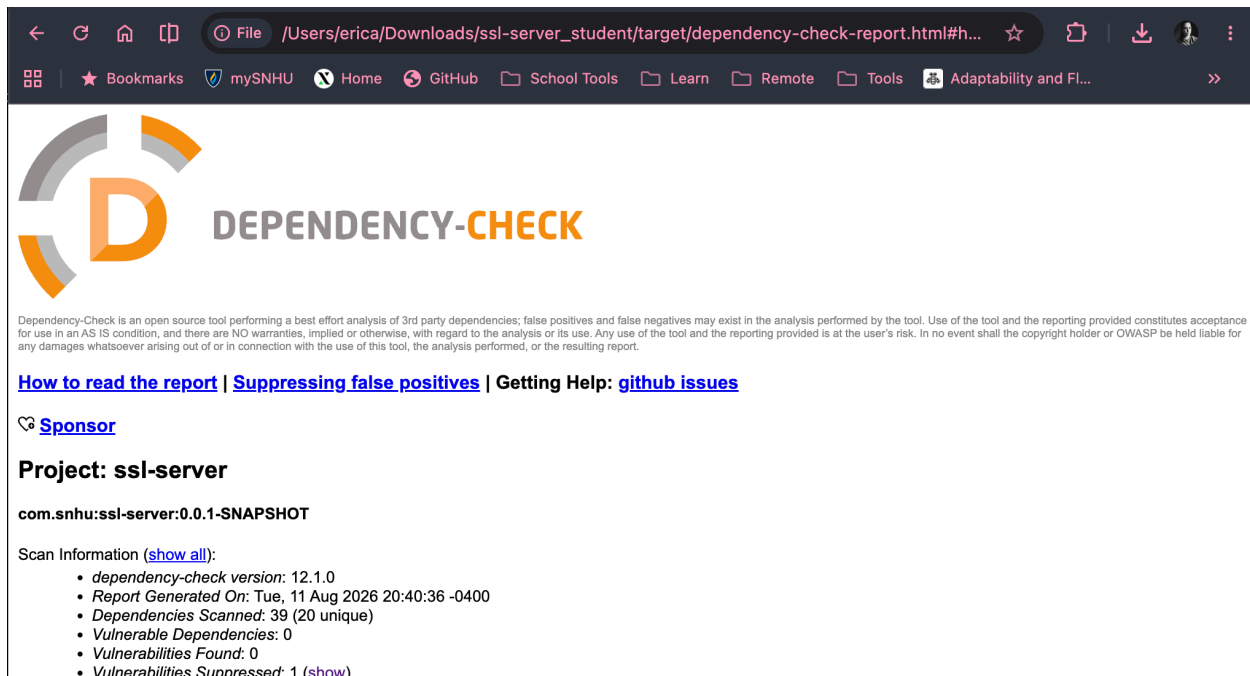


Figure 5: Final OWASP Dependency-Check results after dependency remediation and review.

Functional Testing

I performed functional testing after completing the code and dependency updates. The Maven test and install processes completed with a BUILD SUCCESS result, confirming that the application compiled, passed its automated test, and produced the expected project artifact. I also started the Spring Boot application and confirmed that the embedded server ran with HTTPS enabled on port 8443. The /hash endpoint returned the expected static data and its 64-character SHA-256 checksum.

I manually reviewed the source code for syntactical, logical, and security problems. The syntactical review included the imports, annotations, method declarations, braces, exception handling, and Maven configuration. The successful build confirmed that no unresolved syntax or dependency errors remained.

The logical review covered the data flow through the getHash() method. I verified that the method used the intended static string, converted it with UTF-8, passed the resulting bytes to SHA-256, and converted every digest byte into a two-character hexadecimal value. The leading-zero condition was necessary to ensure that the final digest remained 64 characters long.

The security review confirmed that the application used SHA-256 rather than MD5 or SHA-1, enabled HTTPS, loaded the certificate and private key from a PKCS12 keystore, and used updated dependencies. I also identified a remaining production concern: the local project stores keystore credentials in application.properties. The configuration works for this demonstration, but production credentials should be supplied through protected environment variables or a secrets-management service rather than committed to the source repository.

These tests showed that the security changes did not interfere with the required application behavior. The application generated the checksum, loaded the certificate and keystore, established the HTTPS endpoint, and completed the Maven build without errors.

```
[INFO] --- maven-install-plugin:3.1.4:install (default-install) @ ssl-server ---
[INFO] Installing /Users/erica/Downloads/ssl-server_student/pom.xml to /Users/erica/.m2/repository/com/snhu/ssl-server/0.0.1-SNAPSHOT/ssl-server-0.0.1-SNAPSHOT.pom
[INFO] Installing /Users/erica/Downloads/ssl-server_student/target/ssl-server-0.0.1-SNAPSHOT.jar to /Users/erica/.m2/repository/com/snhu/ssl-server/0.0.1-SNAPSHOT/ssl-server-0.0.1-SNAPSHOT.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 7.486 s
[INFO] Finished at: 2026-08-11T20:40:36-04:00
[INFO] -----
● erica@Ericas-MacBook-Air ssl-server_student % open target/dependency-check-report.html
○ erica@Ericas-MacBook-Air ssl-server_student %
○ erica@Ericas-MacBook-Air ssl-server_student % █
completed. Java: Ready Ln 13, Col 1 Spaces: 4 UTF-8 LF {} XML ⚙️ 🔍 Pre
```

Figure 6: Successful Maven build and functional test after completing the security updates.

Summary

The refactoring added several layers of security to the Artemis Financial application. SHA-256 provides a checksum that can reveal unexpected changes to the static data. The RSA key pair and self-signed X.509 certificate allow the local server to establish a TLS connection. HTTPS protects information while it travels between the browser and server. Dependency analysis and functional testing provide additional checks for vulnerable components, code errors, and unintended changes.

The vulnerability assessment process flow identifies several areas that were addressed during the project. The architecture review included an examination of the application's communication design and dependency structure. The cryptography review addressed the SHA-256 checksum, certificate key pair, and TLS configuration. The client/server review addressed the conversion from HTTP to HTTPS. The code-quality review covered the hash implementation, character encoding, hexadecimal conversion, and build results. The plug-in review used OWASP Dependency-Check to identify third-party components associated with known vulnerabilities.

Security was added in layers rather than through a single control. The checksum supports data integrity, TLS provides confidentiality in transit, and the certificate supports server identification during the connection. Dependency updates reduced exposure to known component vulnerabilities. Manual code review and Maven testing confirmed that the refactored application remained functional. The final Dependency-Check report contained zero active vulnerabilities and one documented suppression for functionality that was not present in the application.

Artemis Financial should continue running dependency scans during each build, reviewing new vulnerability reports, and applying security updates promptly. Before production deployment, the company should replace the self-signed certificate with one issued by a trusted certificate authority and move keystore credentials out of the packaged configuration.

Industry Standard Best Practices

I applied secure coding practices throughout the refactoring process. I used Java's standard MessageDigest API instead of creating a custom hash implementation. I specified UTF-8 so the checksum would be generated consistently across systems. I selected SHA-256 rather than an obsolete hash algorithm, generated a 2,048-bit RSA key pair, stored the private key in a PKCS12 keystore, and enabled HTTPS for the application endpoint.

I also reviewed the project's third-party components with OWASP Dependency-Check. The initial scan revealed outdated libraries, so I updated Spring Boot, Java, Tomcat, and Log4j and removed a dependency that the application did not use. I rebuilt and retested the application after making these changes. The single suppression was limited to one CVE and the specific Tomcat components after I verified that the affected example application was not included. This preserved a record of the decision instead of hiding unrelated findings.

These practices helped maintain the application's existing functionality. The Maven build, Spring Boot startup, HTTPS endpoint, data string, and checksum continued to work after the security changes. Retesting after each group of changes reduced the likelihood that a security update would introduce a syntax error or break the application's required behavior.

NIST recommends incorporating secure development practices throughout the software development life cycle to reduce vulnerabilities and address their underlying causes (Souppaya et al., 2022). Artemis Financial should continue this process through code reviews, automated tests, dependency monitoring, documented security requirements, and security reviews before releases.

The business value of these practices extends beyond the source code. Artemis Financial handles sensitive financial information, so a preventable security incident could expose customer data, interrupt operations, damage customer trust, and create recovery costs. Maintaining current dependencies, encrypted communications, controlled certificates, and documented testing decisions reduces these risks and makes future maintenance easier.

The organization should keep private keys and keystore passwords out of the source repository and application.properties file. Spring Boot supports external configuration through environment variables, system properties, and files outside the packaged application (Spring Boot, 2026). Production secrets should be restricted to authorized services and personnel, monitored for unauthorized use, and rotated according to company policy.

References

- Dworkin, M. (2007). Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC (NIST SP 800-38D). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-38D>
- National Institute of Standards and Technology. (2001). Advanced Encryption Standard (AES) (FIPS PUB 197). <https://doi.org/10.6028/NIST.FIPS.197>
- National Institute of Standards and Technology. (2015). Secure Hash Standard (SHS) (FIPS PUB 180-4). <https://doi.org/10.6028/NIST.FIPS.180-4>
- Oracle. (n.d.). The keytool command. <https://docs.oracle.com/en/java/javase/17/docs/specs/man/keytool.html>
- Oracle. (2017). Java security standard algorithm names. <https://docs.oracle.com/javase/9/docs/specs/security/standard-names.html>
- OWASP Foundation. (2026). OWASP Dependency-Check. <https://owasp.org/www-project-dependency-check/>
- Souppaya, M., Scarfone, K., & Dodson, D. (2022). Secure Software Development Framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities (NIST SP 800-218). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-218>
- Spring Boot. (2026). Externalized configuration. <https://docs.spring.io/spring-boot/reference/features/external-config.html>

Appendix A: Refactored Code Files

SslServerApplication.java — implements the /hash endpoint and SHA-256 checksum.

application.properties — configures HTTPS, port 8443, the certificate alias, and the PKCS12 keystore.

pom.xml — contains updated dependencies and the OWASP Dependency-Check plug-in.

keystore.p12 — contains the certificate and private key used for local TLS.

sslserver.cer — exported public X.509 certificate.

dependency-check-suppression.xml — documents the reviewed Tomcat finding.